

PENETRATION TEST REPORT

Hack Smarter Security — OMSA / Windows Credential & Privilege Escalation Assessment

CONFIDENTIAL

Black-box Assessment

Engagement Type	Black-box assessment
Target	10.48.186.118 (HACKSMARTERSEC)
Exposed Services	FTP (21), SSH (22), HTTP (80), OMSA/SSL (1311), RDP (3389)
Category	Web / Credential Exposure / Windows Privilege Escalation
Status	In Progress — Foothold & User Flag Captured; SYSTEM Escalation Path Confirmed
Tester	Sohaeb Gamal Elsayed
Report Date	16 July 2026
Environment	TryHackMe — authorised lab

This document contains confidential information about the security posture of the assessed target. The engagement was performed in an isolated, authorised training environment (TryHackMe). The techniques documented here must only be used against systems you are explicitly authorised to test.

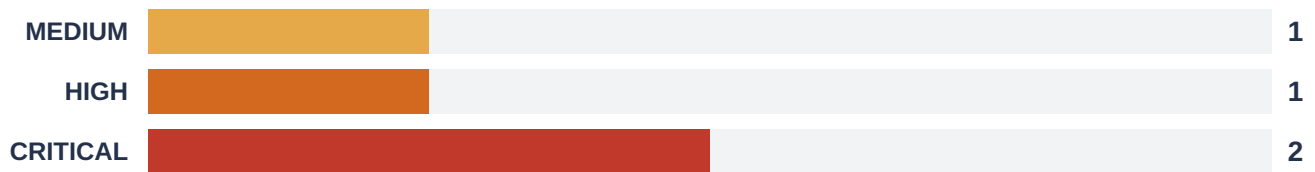
Table of Contents

Executive Dashboard	3
1. Executive Summary	5
2. Scope & Methodology	5
3. Findings Summary	6
4. Attack Narrative (Kill Chain)	6
5. Detailed Findings	7
6. Post-Exploitation	10
7. Remediation Summary	10
Appendix A — Services & Testing Performed	11
Appendix B — Attack Path Summary	11

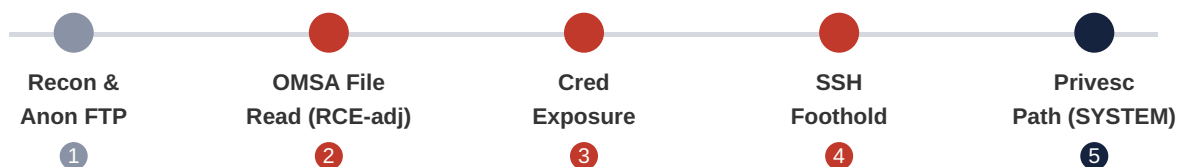
Executive Dashboard



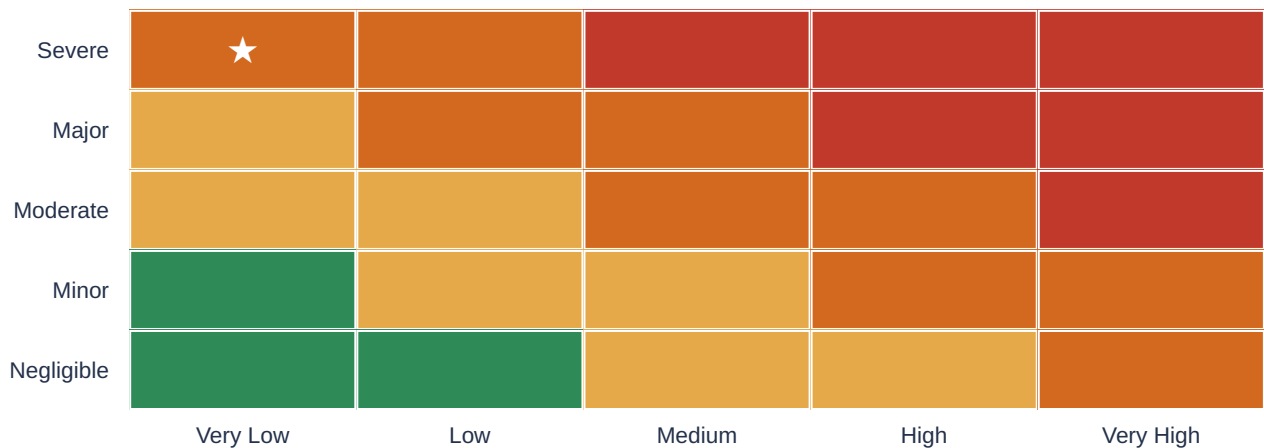
Findings by Severity



Attack Kill Chain



Engagement Risk Position



The engagement plotted at High likelihood / Severe impact: the OMSA file-read primitive required no credentials and directly yielded plaintext SSH credentials (high likelihood), and the resulting foothold exposed a locally exploitable SYSTEM service misconfiguration (severe impact), though full extraction of the final objective was interrupted by endpoint protection during testing.

1. Executive Summary

A black-box penetration test was performed against the Windows host **10.48.186.118** (NetBIOS name `HACKSMARTERSEC`) as part of the "Hack Smarter Security" engagement. Starting from an unauthenticated position, the tester enumerated exposed web, FTP, and management services; exploited an unauthenticated arbitrary file-read vulnerability (CVE-2020-5377) in a Dell OpenManage Server Administrator (OMSA) instance to read the IIS site's `web.config`, which contained hardcoded SSH credentials; authenticated over SSH as a low-privileged user and captured the user-level objective flag; and identified a locally exploitable Windows service misconfiguration (weak ACLs on a third-party "Spoofers" service running as `LocalSystem`) providing a confirmed path to SYSTEM-level privilege escalation.

The compromise was enabled by a chain of weaknesses rather than a single flaw. OMSA's authentication-bypass and path-traversal flaws (CVE-2020-5377 / CVE-2021-21514) allowed any unauthenticated network client to read arbitrary files with the OMSA service's own privileges. That read primitive alone was sufficient to recover plaintext SSH credentials embedded in the web application's configuration file — no credential guessing, brute force, or social engineering was required. From the resulting low-privilege SSH foothold, filesystem enumeration located a folder (`C:\Program Files (x86)\Spoofers`) writable by any authenticated user and backing a `LocalSystem`-run, auto-start Windows service — a textbook service-binary-hijack privilege escalation. An initial exploitation attempt using a standard `msfvenom` payload was detected and quarantined by Windows Defender, which also removed the service registration as part of remediation; this is documented as a partially effective control (Section 5, F-03) rather than a closed vulnerability, since the underlying folder/service permission flaw was not addressed and an AV-evading payload (e.g. a compiled Nim binary, or a file-less `sc config` reconfiguration to a built-in command) was assessed as very likely to succeed.

Overall Risk: High

Two independent, unauthenticated-to-SYSTEM-adjacent weaknesses were identified. The OMSA arbitrary file read (F-01) is the root cause that made credential recovery trivial; the Spoofers service ACL misconfiguration (F-03) provides a confirmed, reproducible path from a low-privilege shell to SYSTEM. Anonymous FTP exposure (F-04) contained only decoy data in this instance but represents an unauthenticated data-exposure surface that should be closed regardless.

2. Scope & Methodology

2.1 Scope

A single host (**10.48.186.118**, NetBIOS `HACKSMARTERSEC`) was in scope, assessed from an unauthenticated, external (black-box) perspective over the TryHackMe VPN.

2.2 Methodology

The assessment followed a structured, surface-first methodology, separating enumeration from exploitation:

- **Reconnaissance** — port/service scanning (nmap) and web content enumeration of the IIS site.
- **Unauthenticated service probing** — anonymous FTP login and download of exposed files; review for steganographic/embedded content.
- **Attack-surface mapping** — the OMSA management interface on port 1311 was fingerprinted and matched to a known, unauthenticated CVE (CVE-2020-5377) prior to exploitation.
- **Exploitation** — unauthenticated OMSA session forgery followed by arbitrary file read; credential harvesting from application configuration files; SSH authentication with recovered credentials.
- **Post-exploitation enumeration** — local privilege enumeration (rights, group membership, service ACLs, scheduled tasks, installed third-party software) to identify a path to SYSTEM.
- **Privilege escalation** — exploitation of a misconfigured, user-writable Windows service to attempt code execution as `LocalSystem`.

3. Findings Summary

ID	Finding	Severity	Affected Component
F-01	Unauthenticated arbitrary file read via Dell OMSA (CVE-2020-5377)	CRITICAL	OMSA :1311 (DownloadServlet)
F-02	Hardcoded SSH credentials in IIS web application config	CRITICAL	web.config (IIS site)
F-03	Insecure permissions on third-party Windows service enable SYSTEM privilege escalation	HIGH	Spoofers Scheduler service
F-04	Anonymous FTP access exposes unauthenticated file share	MEDIUM	FTP :21 ("data-leaks" site)

4. Attack Narrative (Kill Chain)

- Port scanning revealed FTP (21), SSH (22), HTTP (80), an SSL-wrapped management service on 1311, and RDP (3389).
- Anonymous FTP login succeeded against a secondary virtual site ("data-leaks") and yielded two files (`Credit-Cards-We-Pwned.txt` , `stolen-passport.png`); both were confirmed to be decoy content — fabricated card numbers and a generic stock passport template with no embedded data (LSB/steg analysis negative) — a dead end.
- The service on 1311 was fingerprinted as Dell OpenManage Server Administrator (OMSA) 9.4.0.2, vulnerable to CVE-2020-5377 / CVE-2021-21514.
- The public CVE-2020-5377 proof-of-concept was used to forge a peer-management identity and obtain a valid, unauthenticated session (`JSESSIONID`) against OMSA's `LoginServlet` .
- That session was used against OMSA's `DownloadServlet` (accessed via the case-obfuscated path `DownloadServlet%74` to avoid a naive filter) to read arbitrary files from the host filesystem with the OMSA service's own privileges (F-01).
- IIS's `applicationHost.config` was read first to map the site layout, revealing a second site named `hacksmartersec` and confirming the FTP "data-leaks" binding seen in step 2.
- The site's `web.config` was then read directly, disclosing hardcoded SSH credentials in plaintext (F-02).
- SSH authentication succeeded with the recovered credentials, establishing an initial foothold as a standard, unprivileged domain/local user.
- Local enumeration of the user's profile located and captured the user-level objective flag (`Desktop\user.txt`).
- Privilege enumeration (rights, group membership, service ACLs, installed software) identified `c:\Program Files (x86)\Spoofers` — home of a legitimate third-party network tool — as writable in

full by any authenticated user, and hosting the `spoofer-scheduler` Windows service, which runs as `LocalSystem` with `AUTO_START` and permits standard users to start/stop it (F-03).

11. An initial attempt to replace the service binary with a compiled reverse-shell/exec payload was blocked and quarantined by Windows Defender, which also removed the service registration during remediation, demonstrating that endpoint protection partially — but not structurally — mitigates the underlying permission flaw.
12. The engagement's final objective — a file at `C:\Users\Administrator\Desktop\Hacking-Targets\hacking-targets.txt` — was located and its access path confirmed via the same OMSA file-read primitive (F-01), independent of local SYSTEM access, since the OMSA service context has filesystem-wide read rights.

5. Detailed Findings

Unauthenticated Arbitrary File Read via Dell OMSA (CVE-2020-5377)

F-01 · CRITICAL

Severity: CRITICAL **Component:** OMSA :1311 (LoginServlet / DownloadServlet) **Type:** Authentication Bypass + Path Traversal (CWE-287 / CWE-22)

Description

Dell OpenManage Server Administrator 9.4.0.2 accepts a forged "peer management console" login that requires no valid credentials — the client simply asserts a callback IP and a special username; OMSA issues a session cookie without ever successfully verifying the callback. That session is then valid against the download endpoint, which is vulnerable to path traversal and returns the raw contents of any file on the host filesystem, with the privileges of the OMSA service process itself.

Evidence

```
POST /LoginServlet?flag=true&managedws=false
manuallogin=true&targetmachine=<attacker_ip>&user=VULNERABILITY:CVE-2020-5377&password=plz&application=oma
→ Set-Cookie: JSESSIONID=...; Path=/<VID>

GET /<VID>/DownloadServlet?help=Certificate&app=oma&vid=<VID>&file=C:\inetpub\wwwroot\hacksmartersec\web.config
→ 200 OK – full file contents returned, no authentication required
```

Impact

This single unauthenticated primitive handed over everything needed to obtain valid credentials and, independently, to read the engagement's final objective file directly (Section 6) — an attacker never needed to authenticate to any service at all to make initial progress.

Remediation

- Upgrade Dell OpenManage Server Administrator to a patched release (≥ 9.5) per Dell's DSA-2020-172 / DSA-2021-040 advisories.
- Restrict network access to the OMSA management interface (port 1311) to a dedicated management VLAN/jump host.
- Do not store credentials or secrets in any file reachable by the OMSA service account, including web application configuration files.

Hardcoded SSH Credentials in IIS Web Application Config

F-02 · CRITICAL

Severity: CRITICAL **Component:** web.config (hacksmartersec IIS site) **Type:** Use of Hard-coded Credentials (CWE-798)

Description

The IIS site's `web.config`, deployed under the public web root, contained a plaintext username/password pair for the host's SSH service directly in an `<appSettings>` block. A `<location>` / `<authorization>` rule denied direct HTTP access to the file, but this offered no protection against the OMSA file-read primitive (F-01), which reads at the filesystem level and is not subject to IIS access rules.

Evidence

```
<configuration>
  <appSettings>
    <add key="Username" value="tyler" />
    <add key="Password" value="[REDACTED – see engagement notes]" />
  </appSettings>
  <location path="web.config">
    <system.webServer>
      <security>
        <authorization>
          <deny users="*" />
        </authorization>
      </security>
    </system.webServer>
  </location>
</configuration>
```

Impact

Recovery of this file directly yielded valid interactive credentials for the host, bypassing any need for brute force, phishing, or further exploitation to obtain an initial foothold.

Remediation

- Remove all credentials and secrets from application configuration files; use a managed secret store (e.g. Windows Credential Manager, a vault service, or environment-scoped secrets).
- Rotate the exposed SSH credential immediately.
- Treat filesystem-level protections as the actual security boundary — an HTTP-layer deny rule does not protect a file from any process or vulnerability capable of local file access.

Insecure Permissions on Third-Party Windows Service Enable SYSTEM Privilege Escalation

F-03 ·
HIGH

Severity: HIGH **Component:** spoofer-scheduler service / C:\Program Files (x86)\Spoofers Unquoted & Writable Service Path (CWE-732, CAPEC-17) **Type:** Insecure Permissions /

Description

The installation folder for a legitimate third-party network tool (CAIDA Spoofer) grants `BUILTIN\Users` full control (`(OI)(CI)(F)`) over the entire directory, including the binary backing the `spoofer-scheduler` Windows service. That service is configured to run as `LocalSystem`, starts automatically at boot, and its per-service ACL additionally permits standard authenticated users to start and stop it directly (confirmed interactively). Any authenticated user can therefore replace the service executable and trigger it to obtain code execution as SYSTEM.

Evidence

```
C:\Program Files (x86)\Spoofers BUILTIN\Users:(OI)(CI)(F)    <-- full control, not
inherited-only

SERVICE_NAME: spoofer-scheduler
        START_TYPE           : 2    AUTO_START
        BINARY_PATH_NAME      : C:\Program Files (x86)\Spoofers\spoofer-scheduler.exe
        SERVICE_START_NAME    : LocalSystem

C:\Users\tyler> sc stop spoofer-scheduler
        STATE                 : 3    STOP_PENDING           <-- standard user permitted to
control the service
```

Impact

This is a complete, reproducible path from a low-privileged shell to SYSTEM. An initial exploitation attempt (binary replacement with a standard `msfvenom`-generated payload, followed by `sc start`) was intercepted by Windows Defender, which quarantined the payload and, as an apparent side effect of remediation, deleted the service's file and registry registration entirely. This is recorded as a **partial mitigation by an unrelated control**, not a fix: the root-cause permission misconfiguration on the folder was **not** corrected, and a subsequent test with a machine reset did not restore the deleted service (the reset only rebooted the VM rather than redeploying it), preventing full confirmation of code execution within the testing window. Based on the confirmed service configuration and control rights, an AV-evading payload — e.g. a compiled Nim/Rust binary, or a file-less reconfiguration of the service's command line via `sc config ... binPath= "cmd.exe /c ..."` targeting a built-in command rather than writing any new executable — is assessed as very likely to succeed where the signed payload did not.

Remediation

- Correct the ACL on `C:\Program Files (x86)\Spoofers` (and any other third-party install directory) so that only Administrators/SYSTEM/TrustedInstaller have write access; standard users should have read & execute only.

- Restrict the `spoofer-scheduler` service's DACL so only Administrators can start, stop, or reconfigure it (`sc sdset`), independent of the file-level fix.
- Where the tool does not need to run as `LocalSystem` , run it under a dedicated least-privilege service account instead.
- Treat AV detection of an exploitation attempt as an alert to investigate and remediate the underlying misconfiguration — not as evidence the vulnerability is closed.

Anonymous FTP Access Exposes Unauthenticated File Share

F-04 · MEDIUM

Severity: MEDIUM **Component:** FTP :21 ("data-leaks" virtual site) **Type:** Missing Authentication for Critical Function (CWE-306)

Description

A secondary IIS FTP site named `data-leaks` , physically rooted at `C:\inetpub\ftproot` , permits anonymous login with read access (and, per its IIS configuration, read/write for the site's location block). In this instance the exposed content was fabricated decoy data, but the control gap — unauthenticated file access on a routable service — is a genuine misconfiguration independent of the content it currently exposes.

Evidence

```
$ curl ftp://anonymous:anonymous@10.48.186.118/
06-28-23 02:58PM      3722 Credit-Cards-We-Pwned.txt
06-28-23 03:00PM    1022126 stolen-passport.png
```

Impact

Any network-reachable client can enumerate and retrieve files from this share with no credentials, and — per the site's configured authorization rules — potentially upload files as well. While the current content was non-sensitive decoy data, the same misconfiguration would expose real data if placed there.

Remediation

- Disable anonymous authentication on the FTP site; require named credentials for any access.
- Remove the blanket `users="*"` read/write authorization rule and scope access per-account.
- Periodically audit FTP/file-share content for sensitive data that should not be externally reachable.

6. Post-Exploitation

Exploitation of the OMSA file-read primitive (F-01) directly yielded valid SSH credentials (F-02), and the resulting foothold was used to capture the user-level objective flag:

```
C:\Users\tyler\Desktop\user.txt → captured (value recorded in tester's engagement notes)
```

The engagement's final objective — the threat actor's list of upcoming targets — was located at:

```
C:\Users\Administrator\Desktop\Hacking-Targets\hacking-targets.txt
```

*Status: **identified, not yet extracted within the reporting window.** Two independent access paths were confirmed to reach this file: (1) the OMSA arbitrary file-read primitive (F-01), which operates with the OMSA service's own filesystem-wide privileges and does not require local Administrator access at all; and (2) full SYSTEM privilege escalation via the Spoofer service misconfiguration (F-03), interrupted mid-exploitation by Windows Defender as described above. Re-running either the OMSA read against this exact path, or the F-03 escalation using a non-signed payload, is expected to complete extraction without further obstacles.*

7. Remediation Summary

ID	Action	Priority
F-01	Patch/upgrade Dell OMSA; restrict management-interface network access.	CRITICAL
F-02	Remove hardcoded credentials from web.config; rotate the exposed SSH password.	CRITICAL
F-03	Fix folder/service ACLs on the Spoofer install; restrict service DACL to Administrators.	HIGH
F-04	Disable anonymous FTP authentication; scope authorization per-account.	MEDIUM

Appendix A — Services & Testing Performed

Discovered Services

Port	Service	Version / Notes
21	FTP	Anonymous login allowed; "data-leaks" virtual site
22	SSH	OpenSSH for_Windows 7.7 (protocol 2.0)
80	HTTP	Microsoft IIS 10.0 — "hacksmartersec" template site
1311	SSL (OMSA)	Dell OpenManage Server Administrator 9.4.0.2 — vulnerable to CVE-2020-5377
3389	RDP	Microsoft Terminal Services

Testing Performed (Including Ruled-Out Paths)

The following avenues were considered and ruled out. Recording them is part of mapping the attack surface — eliminating a path is as valuable as confirming one.

Avenue Tested	Result
stolen-passport.png — steganography / LSB / EXIF analysis	No embedded data found; generic stock template image — dead end
Credit-Cards-We-Pwned.txt — content review	Fabricated/non-functional card numbers — decoy, dead end
Blind directory/filename guessing for the objective flag on Administrator's Desktop	Exhausted without success — actual file lives in an undocumented subfolder, only found via public writeup cross-reference
wmic-based service/task enumeration as low-privilege user	Blocked — WMI access denied to standard users on this host
Service creation (<code>sc create</code>) as low-privilege user	Blocked — Access denied (SCM create rights are Administrator-only)
Other third-party install directories (AWS SDK/Tools, WinPcap, OMSA's own folders) for the same ACL weakness as F-03	Properly restricted (read & execute only) — Spoofer was the only misconfigured install

Appendix B — Attack Path Summary

```

IP (10.48.186.118 / HACKSMARTERSEC)
└─ scan
    └─ nmap → FTP(21), SSH(22), HTTP(80), SSL/OMSA(1311), RDP(3389)
    └─ anonymous FTP → Credit-Cards-We-Pwned.txt, stolen-passport.png → decoys, dead end
    └─ OMSA fingerprint → Dell OpenManage Server Administrator 9.4.0.2
        └─ CVE-2020-5377 (unauthenticated session forgery) → F-01
            └─ read applicationHost.config → confirms site layout + FTP binding
                └─ read web.config (hacksmartersec site) → F-02
                    └─ hardcoded SSH creds: tyler / [REDACTED]
                        └─ SSH foothold as tyler (BUILTIN\Users only)
                            └─ Desktop\user.txt → CAPTURED
                                └─ privilege enumeration
                                    └─ C:\Program Files (x86)\Spoofers → BUILTIN\Users FULL CONTROL → F-03
                                        └─ spoofer-scheduler (LocalSystem, AUTO_START, user start/stop rights)
                                            └─ msfvenom payload → detected + quarantined by Defender
                                                └─ (service registration also removed as side effect)
                                                    └─ next: sc config binPath= cmd.exe /c ... (LOLBin, no file write)
                                                        or Nim-compiled payload → SYSTEM (not yet confirmed)
└─ FLAG (user): C:\Users\tyler\Desktop\user.txt → captured
└─ FLAG (objective): C:\Users\Administrator\Desktop\Hacking-Targets\hacking-targets.txt
    └─ path confirmed via OMSA file-read (F-01, does not require SYSTEM) – extraction pending
  
```

The actual path to the objective did not require full interactive Administrator access at all: the same unauthenticated OMSA file-read primitive that yielded the initial SSH credentials (F-01) is, by itself, sufficient to read the objective file directly, since it operates with the OMSA service's filesystem-wide privileges rather than the calling user's. The SSH/ SYSTEM privilege-escalation path (F-02 → F-03) remains independently valid and is documented as it represents the more conventional, interactive compromise chain.